

Phase 1 Infra Validation and Phase 2 CI/CD

Final synchronized AWS validation, pipeline and rollback checklist

Flipkart Clone Microservices Workspace

Phase 1 Infrastructure Validation and Phase 2 CI/CD Setup

This is the final synchronized checklist for the Flipkart Clone AWS deployment.

Use this **after your local code works with Docker Compose** and after the first AWS infrastructure deployment is created.

Project path:

```
/home/user/flipkart-clone
```

Important project-specific corrections:

- Products API route is `/api/products`, not `/api/products/products`.
- Gateway health can be tested at `/health` or `/api/health`.
- User service health through gateway is `/api/users/health`.
- Frontend should use `REACT_APP_API_URL=/api` when served behind ALB with `/api/*` listener rule.
- AWS backend images should use `Dockerfile.aws` so DocumentDB TLS CA bundle exists at `/app/global-bundle.pem`.

PHASE 1: Infrastructure Testing and Validation

Do this **before setting up CI/CD**. Only proceed to Phase 2 when all checks pass.

Step 1: Verify VPC and Networking

Console path:

```
VPC → Your VPC → flipkart-vpc
```

Check	How to Verify	Expected Result
VPC created	VPC Dashboard → Status	Available
Subnets	Subnets tab	2 public and 2 private, all Available
Route tables	Route Tables tab	Public route → Internet Gateway, private route → NAT Gateway if created
Internet Gateway	Internet Gateways tab	Attached to flipkart-vpc
NAT Gateway	NAT Gateways tab	Available if using private subnets with outbound internet

Test Private Subnet Internet Access

If you created a bastion EC2 in a public subnet:

```
ssh -i key.pem ec2-user@BASTION_PUBLIC_IP
curl -I https://aws.amazon.com
```

Expected:

```
HTTP 200, 301, or 302 response
```

If this fails:

- Check private route table points to NAT Gateway.
- Check NAT Gateway is in a public subnet.
- Check public subnet route table points to Internet Gateway.
- Check security group and network ACLs.

Step 2: Verify Security Groups

Console path:

VPC → Security Groups

Security Group	Inbound Rules	Source
flipkart-alb-sg	80, 443	0.0.0.0/0
flipkart-api-gateway-sg	3000	flipkart-alb-sg
flipkart-services-sg	3001-3006	flipkart-api-gateway-sg
flipkart-services-sg	3001-3006 or all TCP	flipkart-services-sg itself
flipkart-docdb-sg	27017	flipkart-services-sg
flipkart-redis-sg	6379	flipkart-services-sg

Important self-reference rule:

flipkart-services-sg should allow service-to-service traffic from itself.

This is needed because:

- Cart service calls Product service.
- Order service calls Cart and Notification services.
- Payment service calls Order and Notification services.

Step 3: Verify DocumentDB

Console path:

DocumentDB → Clusters → flipkart-docdb-cluster

Check	Expected
Cluster status	Available
Instances	1 or 2 instances, all Available
Endpoint	Cluster endpoint copied
Security group	flipkart-docdb-sg attached
Backup	Enabled according to your retention setting

Correct DocumentDB URI for This Project

Use this pattern in ECS task definitions:

```
mongodb://docdbadmin:URL_ENCODED_PASSWORD@DOCDB_ENDPOINT:27017/flipkart-  
users?tls=true&tlsCAFile=/app/global-  
bundle.pem&replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false
```

Change database name per service:

```
flipkart-users  
flipkart-products  
flipkart-carts  
flipkart-orders  
flipkart-payments  
flipkart-notifications
```

Important:

- Use `Dockerfile.aws` for backend images.
- URL-encode special characters in password.
- Keep `retryWrites=false` for DocumentDB.

Test Connection from Bastion

```
mongosh "mongodb://docdbadmin:PASSWORD@YOUR_DOCDB_ENDPOINT:27017/?tls=true&replicaSet=rs0&readPr  
eference=secondaryPreferred&retryWrites=false" --tls --tlsAllowInvalidHostnames
```

Inside Mongo shell:

```
show dbs  
use flipkart-users  
db.users.find().limit(1)
```

Expected:

Connection succeeds without timeout or authentication errors.

Step 4: Verify ElastiCache Redis Optional

Console path:

ElastiCache → Redis clusters → flipkart-redis

Check	Expected
Status	Available
Nodes	Primary and replica available if configured
Endpoint	Primary endpoint copied
Security group	flipkart-redis-sg attached

Test connection from bastion:

```
sudo yum install redis6 -y || sudo yum install redis -y  
redis-cli -h YOUR_REDIS_ENDPOINT -p 6379 --tls ping
```

Expected:

PONG

If Redis is not implemented yet in app code, this check is optional. The current platform runs without Redis.

Step 5: Verify ECS Cluster and Services

Console path:

ECS → Clusters → flipkart-cluster

Check	Expected
Cluster status	ACTIVE
Services	All 8 services show ACTIVE
Running tasks	Each service has desired count running, for example 2/2
Task status	RUNNING, not stuck in PENDING or STOPPED
Deployments	Latest deployment stable

Services:

frontend-service
api-gateway-service
user-service
product-service
cart-service
order-service
payment-service
notification-service

Check Container Logs

1. Click service.
2. Open **Tasks** tab.
3. Click running task.
4. Open **Logs** tab.
5. Look for startup messages like:

User Service DB connected
Product Service DB connected
Cart Service DB connected
Order Service DB connected
Payment Service DB connected
Notification Service DB connected
API Gateway running on port 3000

No repeated errors should appear.

Common ECS Log Errors and Fixes

Error	Likely Cause	Fix
MongoServerSelectionError	Cannot reach DocumentDB	Check DocDB SG, URI, TLS, private subnets
ECONNREFUSED 3001	Service discovery/security group problem	Verify Cloud Map and service SG self-reference

Error	Likely Cause	Fix
CannotPullContainerError	ECR image or IAM problem	Check repo name, image tag, task execution role
Health checks failed	App not responding or wrong path	Check port, health path, grace period
ResourceInitializationError	Logs/secrets/network issue	Check CloudWatch logs permissions and VPC outbound

Recommended ECS health check grace period:

120 seconds

Step 6: Verify ALB and Routing

Console path:

EC2 → Load Balancers → flipkart-alb

Check	Expected
ALB status	Active
DNS name	Available and copied
Listener HTTP:80	Default forwards to frontend target group
Listener rule	/api/* forwards to API target group
API target group	Targets healthy
Frontend target group	Targets healthy

Target groups:

flipkart-api-tg → api-gateway-service port 3000
 flipkart-frontend-tg → frontend-service port 80

Test Endpoints

Set variable:

ALB="http://YOUR_ALB_DNS"

Frontend:

curl -I \$ALB/

Expected:

HTTP 200 OK

Gateway health:

curl \$ALB/api/health

User service health through gateway:

curl \$ALB/api/users/health

Products API:

```
curl "$ALB/api/products?limit=2"
```

Expected:

JSON response with products array

If products are empty, seed products using a one-off ECS task or connect to product service and run the seed command.

Step 7: End-to-End Application Test

Open browser:

```
http://YOUR_ALB_DNS
```

Test Step	Expected Result
Load homepage	Page loads, products display, no browser console errors
Register user	User account created and logged in or redirected properly
Login	User logged in, username shown
Browse products	Search and category filters work
Add to cart	Cart count updates and item persists
Checkout	Address/payment form loads
Place order	Order created successfully
Orders page	New order appears
Notifications	Order placed notification appears

Verify Order in DocumentDB

```
use flipkart-orders
db.orders.find().sort({createdAt:-1}).limit(1)
```

Expected:

Your latest test order is visible.

Step 8: Verify Monitoring and Logs

Console path:

CloudWatch → Logs → Log groups

Log Group	Should Contain
/ecs/flipkart-api-gateway	Gateway request/proxy logs
/ecs/flipkart-user-service	Auth and DB startup logs
/ecs/flipkart-product-service	Product queries and DB startup logs

Log Group	Should Contain
/ecs/flipkart-cart-service	Cart requests
/ecs/flipkart-order-service	Order placement/status logs
/ecs/flipkart-payment-service	Payment simulation logs
/ecs/flipkart-notification-service	Notification logs
/ecs/flipkart-frontend	Nginx/frontend logs

Check for repeated:

```

ERROR
FATAL
ECONNREFUSED
MongoServerSelectionError
CannotPullContainerError

```

CloudWatch Dashboard

Console path:

CloudWatch → Dashboards → Flipkart-Monitoring

Widgets should show:

- ECS CPU
- ECS Memory
- ALB requests
- ALB target response time
- ALB 4XX/5XX
- DocumentDB CPU/connections
- Redis metrics if Redis is used

Phase 1 Infrastructure Validation Checklist

Mark all before proceeding to CI/CD:

```

[ ] VPC with 2 public and 2 private subnets
[ ] Internet Gateway working
[ ] NAT Gateway working if private tasks need outbound internet
[ ] ALB security group configured
[ ] API Gateway security group configured
[ ] Services security group configured including self-reference
[ ] DocumentDB security group configured
[ ] Redis security group configured if Redis is used
[ ] DocumentDB cluster available and connectable
[ ] Redis available and returns PONG if used
[ ] ECS cluster ACTIVE
[ ] 8 ECS services ACTIVE
[ ] Desired tasks equal running tasks
[ ] No repeated critical CloudWatch log errors
[ ] ALB active
[ ] API target group healthy
[ ] Frontend target group healthy
[ ] ALB / route goes to frontend
[ ] ALB /api/* route goes to API Gateway
[ ] Frontend loads in browser
[ ] User registration works
[ ] Login works

```



```
[ ] Products display using /api/products
[ ] Cart add/remove works
[ ] Checkout creates order
[ ] Order appears in DocumentDB
[ ] CloudWatch metrics populate
[ ] Budget alert configured
```

If all checks pass, proceed to Phase 2.

PHASE 2: CI/CD Pipeline Setup

Now that infrastructure is proven stable, automate deployments.

This workspace already includes two CI/CD approaches:

1. **Recommended:** Single matrix GitHub Actions workflow:

```
.github/workflows/deploy-to-aws-ecs.yml
```

2. **Optional:** Per-service workflow generator:

```
scripts/create-github-workflows.sh
```

Do not enable both automatic approaches at the same time unless you intentionally want duplicate deployments.

Step 1: Prepare GitHub Repository

```
cd /home/user/flipkart-clone

git init
git add .
git commit -m "Initial commit - Flipkart microservices"

git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/flipkart-microservices.git
git push -u origin main
```

Step 2: Add AWS Secrets to GitHub

GitHub path:

Repository → Settings → Secrets and variables → Actions

Add these secrets:

Secret Name	Value
AWS_ACCESS_KEY_ID	IAM access key
AWS_SECRET_ACCESS_KEY	IAM secret key
AWS_REGION	us-east-1
AWS_ACCOUNT_ID	12-digit account ID
ECS_CLUSTER	flipkart-cluster
REACT_APP_API_URL	/api

Security recommendation:

- For production, prefer GitHub OIDC with IAM role instead of long-lived access keys.

Step 3: Confirm Baseline ECS Task Definitions Exist

CI/CD updates existing services and task definitions. Create baseline task definitions first.

Required families:

```
flipkart-api-gateway
flipkart-user-service
flipkart-product-service
flipkart-cart-service
flipkart-order-service
flipkart-payment-service
flipkart-notification-service
flipkart-frontend
```

Task definition templates are included here:

```
devops/ecs-task-definitions/
```

Replace placeholders before using:

```
<ACCOUNT_ID>
<AWS_REGION>
<DOCDB_CLUSTER_ENDPOINT>
<URL_ENCODED_PASSWORD>
<JWT_SECRET>
```

Step 4: Use Existing GitHub Actions Workflow Recommended

The workspace already has:

```
.github/workflows/deploy-to-aws-ecs.yml
```

This workflow:

- Builds images for all or selected services.
- Uses `Dockerfile.aws` when present.
- Pushes images to ECR.
- Forces ECS service deployment.
- Waits for stability.

Manual run:

```
GitHub → Actions → Deploy Flipkart Clone to AWS ECS → Run workflow
```

Select:

```
all
api-gateway
user-service
product-service
cart-service
order-service
payment-service
notification-service
frontend
```

Automatic run:

Push to main branch with changes under service folders.

Step 5: Optional Per-Service Workflow Generation

If you prefer one workflow file per service, use:

```
cd /home/user/flipkart-clone
./scripts/create-github-workflows.sh
```

This generates:

```
.github/workflows/deploy-api-gateway.yml
.github/workflows/deploy-user-service.yml
.github/workflows/deploy-product-service.yml
.github/workflows/deploy-cart-service.yml
.github/workflows/deploy-order-service.yml
.github/workflows/deploy-payment-service.yml
.github/workflows/deploy-notification-service.yml
.github/workflows/deploy-frontend.yml
```

Important:

- If you use generated per-service workflows, disable or remove the matrix workflow to avoid duplicate deployments.
- The generator is corrected for this project and uses the correct ECS service names and Dockerfile.aws support.

Step 6: Push Workflows and Test Manual Deployment

```
git add .github/workflows/ scripts/ docs/ devops/
git commit -m "Add AWS validation and CI/CD workflows"
git push origin main
```

Test:

1. Open GitHub Actions.
2. Run deploy workflow manually for api-gateway.
3. Watch steps:
 - Checkout
 - AWS auth
 - ECR login
 - Docker build/push
 - ECS deployment
 - ECS stability wait

4. Verify:

```
curl http://YOUR_ALB_DNS/api/health
curl http://YOUR_ALB_DNS/api/users/health
```

Step 7: Test Auto Trigger and Zero Downtime

Make a harmless code change:

```
cd /home/user/flipkart-clone
printf "\n// CI/CD test %s\n" "$(date)" >> api-gateway/server.js

git add api-gateway/server.js
git commit -m "Test api gateway auto deploy"
git push origin main
```

Expected:

1. GitHub detects change under `api-gateway/**`.
2. Workflow builds new API Gateway image.
3. Image is pushed to ECR.
4. ECS starts new task.
5. ALB health check passes.
6. ECS drains old task.
7. App remains available.

After test, remove the harmless comment if you want:

```
git revert HEAD
```

Step 8: Pipeline Monitoring and Rollback

View Deployment Logs

- GitHub → Actions → Workflow run → Expand steps
- ECS → Service → Deployments tab
- CloudWatch Logs → Latest task log stream

Rollback Options

Option 1: Re-run a previous successful GitHub Actions workflow.

Option 2: ECS Console rollback if available.

Option 3: CLI rollback:

```
aws ecs update-service \
  --cluster flipkart-cluster \
  --service api-gateway-service \
  --task-definition flipkart-api-gateway:PREVIOUS_REVISION \
```

```
--force-new-deployment \
--region us-east-1
```

Final Deployment Sequence

1. Local Docker Compose works
2. AWS infra deployed
3. ECR images pushed
4. ECS baseline services running
5. Phase 1 validation passed
6. GitHub repo and secrets configured
7. CI/CD workflow enabled
8. Manual pipeline test passed
9. Auto-trigger test passed
10. Monitoring and rollback verified
11. Production-ready automated deployment

Troubleshooting Quick Reference

Issue	Cause	Fix
Workflow fails at AWS auth	Wrong GitHub secrets	Verify keys and region
ECR push denied	IAM lacks ECR permissions	Attach ECR push policy
ECS deploy fails	Service or task family mismatch	Check exact service/family names
Health checks fail	Slow startup or wrong path	Increase grace period, verify /health
Duplicate deployments	Matrix workflow and generated workflows both active	Use only one CI/CD method
New task unhealthy	App crash or env var issue	Check CloudWatch logs
API 502 after deploy	Gateway cannot reach services	Check Cloud Map and SG rules
Mongo timeout	DocumentDB SG/TLS/URI issue	Use <code>Dockerfile.aws</code> and correct URI

Final Checklist Before Calling It Done

- [] Phase 1 infra validation fully passed
- [] GitHub secrets added
- [] Baseline ECS task definitions exist
- [] Baseline ECS services are stable
- [] ECR repos contain latest images
- [] GitHub Actions manual run passed
- [] Auto-trigger deploy passed
- [] ALB endpoints still work after deployment
- [] CloudWatch logs have no repeated critical errors
- [] Rollback process understood
- [] Final ZIP downloaded and backed up